BDD is Back!

Why Every AI Project Should Have Feature Files

Chris Harbert

Founder & CEO, Testery | Host, Developers Who Test Podcast

Everyone is Writing Code Now

The question isn't "who writes the code?"
It's "how do we know it does what it's supposed to?"

Developers | **Testers** | **Business** | **AI Agents**

We need a shared language for defining expected behavior

What is BDD?

Collaboration between devs, testers, and product owners

Given a starting context

When an action occurs

Then an expected outcome

Focus on what the system should do, not how

History of BDD

2006
Dan North coins BDD

2008
Cucumber emerges

2010s
Early promise then friction

Now
AI changes everything

"Unnecessary abstraction" ... boilerplate fatigue ... BDD stalled

Until now.

Feature Files

The perfect interface between humans and AI

```
Feature: User Login

  Scenario: Valid credentials
    Given a registered user
    When they enter correct password
    Then they see the dashboard

  Scenario: Invalid credentials
    Given a registered user
    When they enter wrong password
    Then they see an error message
```

S **Structured**

Enough for machines to parse

S **Scoped**

Enough to limit context

R **Readable**

Enough for anyone to write

Step Definitions

Where BDD historically bogged down — AI eliminates the friction

Before AI

Hours of manual wiring
Boilerplate glue code
Maintenance nightmare
Teams gave up

With AI

Generated in seconds
AI writes the glue code
Focus on behavior, not wiring
Demo: Claude Code in action

What once took hours of boilerplate now takes minutes

The Context Window Problem

Entire codebase in prompt

AI loses track
Makes wrong assumptions
Changes unrelated code

Feature file as context

Small, focused scope
Exact behavior specified
AI stays on task

"Instead of dumping your whole codebase into the prompt, hand AI a feature file."

BDD as the Leash

1. **Write Feature**
2. **AI Implements**
3. **Tests Pass**
4. **Confidence**

Feature files define the boundaries of what should change
Step definitions define what "done" looks like
AI can verify its own work by running the scenarios

Demo: AI staying focused with a feature file vs. going off the rails without one

Live Demo

1. Start with a real project — no BDD
2. Use Claude Code to add Cucumber + generate feature files
3. Write a new feature file for a behavior change
4. Let AI implement guided by the feature file
5. Run scenarios — watch tests pass

Pitfalls to Avoid

Don't let AI write your feature files

Humans define behavior

Keep Gherkin business-readable

No technical jargon

Don't skip the collaboration step

BDD is a conversation

One behavior per scenario

Keep scenarios focused

BDD is Back.

AI eliminates the implementation cost

Feature files give AI focus and boundaries

BDD is the shared language we've been missing

Add a .feature file to your next AI-assisted task and see the difference.